

Homework 02: The Receipt That Adds Up

A shop's till has been rounding wrong and nobody can work out why. You are going to write one that does not, which means getting the money type right.

The Task

Create `Receipt.java` as a compact source file. It should:

1. Hold three items with a name and a price: `19.99`, `4.05` and `0.99`.
2. Print each item on its own line.
3. Print the subtotal, a 20% tax amount, and the total.
4. Get the arithmetic exactly right, and get the *formatting* exactly right too.

Target output:

```
Bread      19.99
Milk       4.05
Gum        0.99
-----
Subtotal   25.03
Tax (20%)  5.01
Total      30.04
```

The Money Rule

Do **not** use `double` for the prices. Module 02 showed why: `0.1 + 0.2` is not `0.3`, and a till that is a hundredth of a penny out on every line eventually fails an audit.

Store each price as a `long` counting **cents**, so `19.99` becomes `1999`. Do all the arithmetic in whole cents and convert to text only at the moment you print.

To format cents as currency, integer division and remainder give you both halves:

```
long cents = 405;
String shown = (cents / 100) + "." + (cents % 100);
```

Run that on the milk and see what you get. It is wrong, and working out *why* it is wrong is the most valuable part of this exercise. Four of the six lines on this receipt hit the same bug, so it is not an edge case you can wave away.

The Rounding Decision

20% of 2503 cents is 500.6 exactly. You cannot charge six tenths of a cent, so you have to decide what happens to it, and your code should make that decision visible rather than letting it happen by accident.

Whatever you choose, leave a comment saying what you chose and why.

Then Reproduce Two Traps

Add a second file, `Traps.java`, that demonstrates and explains:

1. **The `==` trap on boxed integers.** Print a comparison that is `true` and one that is `false` using the same kind of value, then print the `.equals` result showing what you actually meant.
2. **String immutability.** Call a String method without keeping the result, print the unchanged value, then do it properly.

Each needs a comment explaining *why* in your own words. Copying the wording from the examples does not count. Explaining it back is the point.

Acceptance Criteria

- ☐ Both files run with `java Receipt.java` and `java Traps.java`
- ☐ No `double` or `float` appears anywhere in `Receipt.java`
- ☐ Tax is computed, not typed in as a literal
- ☐ `4.05` prints as `4.05`, not `4.5`
- ☐ The subtotal, tax and total all print with exactly two decimal places
- ☐ The columns line up
- ☐ A comment states what happens to the fraction of a cent, and why
- ☐ `Traps.java` shows a boxed `==` printing `true` and another printing `false`
- ☐ Every trap carries an explanation in your own words

Stretch

The till now needs a 7.5% rate. Tax on 2503 cents at 7.5% is 187.725 cents.

Implement it, decide what happens to the fraction, and defend the choice in a comment. There is no single right answer, which is exactly why it is worth thinking about. Real payment systems have published rules for this, some of them requiring rounding per line item rather than on the total, which gives a different total again.

Hint, if the formatting is fighting you

`String.format` understands padding and zero-filling:

```
String.format("%-12s %8s", name, amount)    // left-align 12, right-align 8
String.format("%d.%02d", whole, remainder) // %02d forces two digits
```

The second one is the fix for the bug you found earlier.